Finding the Worst Is Harder Than Finding the Best: Directional Asymmetry in LLM Setwise Ranking

Anonymous Author(s)

Abstract

Large Language Models (LLMs) have emerged as effective zero-shot document rerankers. Among the four prompting paradigms—pointwise, pairwise, listwise, and setwise—the setwise approach offers a compelling trade-off between effectiveness and efficiency by asking the LLM to select the most relevant document from a candidate set. However, current setwise methods only extract one piece of ranking information per LLM call: the *best* document. In this paper, we investigate *bidirectional* selection strategies that extract more information from each comparison. We propose three approaches: (1) **Bottom-Up Setwise Ranking**, which reverses the selection to identify the *least* relevant document; (2) **Dual-End Setwise Ranking**, which asks the LLM to simultaneously identify both the most and least relevant documents; and (3) **Bidirectional Ensemble**, which fuses top-down and bottom-up rankings. We pair the dual-end strategy with two new sorting algorithms—cocktail shaker sort and double-ended selection sort—designed for dual-output comparators. Experiments on TREC DL 2019/2020 with nine LLMs spanning three architecture families (Flan-T5, Qwen3, and Qwen3.5, ranging from 780M to 27B parameters) reveal a clearer story than the original "three equally promising directions" framing would suggest. The headline result is one of *directional asymmetry*: the DualEnd family is the strongest overall, with DualEnd-Cocktail achieving the best mean NDCG@10 across both datasets and DualEnd methods (Cocktail or Selection) winning 14 of 18 model–dataset configurations; BottomUp alone is consistently and sometimes catastrophically weaker than TopDown, with 6 Bonferroni-significant losses; and BiDir ensemble does not outperform TopDown (3 Bonferroni-significant losses, 0 wins). Per-query significance tests are conservative: only one DualEnd gain remains Bonferroni-significant on these 43/54-query test sets, so we frame DualEnd as a quality-first refinement, not a free improvement. The mechanism we point to is positional: standalone worst-selection is dominated by a strong recency bias (the model labels the last passage as "worst" 40–63% of the time), while the same model placed in a joint best-and-worst prompt produces a *reversed* primacy-heavy "dual_worst" pattern that is consistent across all 9 models. Joint elicitation therefore changes the model's behavior, not just its output channel. Our information-theoretic analysis shows that dual-end extracts ∼74–90% more ranking information per LLM call at negligible prompt overhead, and a quality–cost Pareto analysis identifies the empty region between TopDown-Bubble and DualEnd-Cocktail as the natural target for selective and bias-aware DualEnd refinements.

CCS Concepts

• **Information systems** → **Rank aggregation**; **Retrieval models and ranking**.

Keywords

LLM-based ranking, setwise prompting, zero-shot reranking, document ranking, position bias

ACM Reference Format:
Anonymous Author(s). 2026. Finding the Worst Is Harder Than Finding the Best: Directional Asymmetry in LLM Setwise Ranking. In . ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities as zero-shot passage rerankers in information retrieval (IR) systems [13, 16, 23]. Given a query and a set of candidate documents retrieved by a first-stage retriever, LLMs can reorder candidates by relevance without any task-specific fine-tuning. Four prompting paradigms have been established: *pointwise* (score each document independently), *pairwise* (compare documents in pairs), *listwise* (rerank a full list), and *setwise* (select the best from a small candidate set) [23].

The **setwise approach** [23] occupies a sweet spot in the effectiveness–efficiency trade-off. By presenting a small set of candidates (e.g., 4 documents) and asking the LLM "*which passage is the most relevant?*", it extracts more comparative information than pairwise (which compares only 2) while requiring shorter prompts than listwise (which presents 10–20). Combined with classical sorting algorithms like heapsort and bubblesort, setwise ranking achieves high effectiveness with significantly fewer LLM calls than pairwise approaches.

However, current setwise methods have a fundamental limitation: **each LLM call extracts only one ranking decision**—the identity of the *most relevant* document. The comparative assessment the LLM performs internally necessarily evaluates all candidates, yet we discard all information except the winner. This is analogous to running a tournament but only recording gold medals while throwing away all other placement information.

Consider a concrete example: when an LLM is presented with four passages and asked "which is the most relevant?", it must internally assess all four before producing an answer. The model effectively *knows* which passage is the worst too—it simply is not asked to report it. From an information-theoretic perspective, selecting the best from m candidates yields $\log_2(m)$ bits of information. But by asking for *both* the best and worst, we obtain $\log_2(m) + \log_2(m-1)$ bits—nearly doubling the information extracted per call with negligible prompt overhead.

In this paper, we ask: *can we extract more ranking information from each LLM comparison?* We investigate three bidirectional strategies:

(1) **Bottom-Up Setwise Ranking (BottomUp)**: We reverse the prompt to ask "*which passage is the least relevant?*" and build the ranking from the bottom up by iteratively eliminating the worst documents. This tests whether LLMs exhibit different

accuracy and bias patterns when identifying irrelevant versus relevant documents.

(2) **Dual-End Setwise Ranking (DualEnd)**: We modify the prompt to ask for *both* the most and least relevant documents simultaneously: "*which is the most relevant and which is the least relevant?*" This extracts two ranking decisions per LLM call. We pair this prompt with two new sorting algorithms—cocktail shaker sort and double-ended selection sort—that are designed to consume both outputs.

(3) **Bidirectional Ensemble (BiDir)**: We run both top-down (standard) and bottom-up ranking independently and fuse the results using rank aggregation methods. This tests whether the two directions provide complementary relevance signals.

Figure 1 illustrates the key differences between standard top-down, bottom-up, and dual-end setwise ranking.

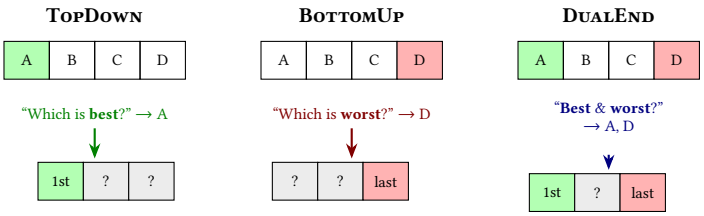

Figure 1: Comparison of setwise ranking strategies. TopDown identifies only the best document per call. BottomUp identifies only the worst. DualEnd identifies both, extracting ~1.79× ranking information per LLM call.

Findings. Across nine LLMs (Flan-T5 780M–11B, Qwen3 4B–14B, Qwen3.5 4B–27B) and the TREC Deep Learning 2019/2020 benchmarks, we observe a clearer story than the original "three equally promising directions" framing would suggest. The headline result is one of *directional asymmetry*: worst-selection alone is not a mirror image of best-selection. Specifically:

- The DualEnd family is the strongest overall. DualEnd-Cocktail achieves the best mean NDCG@10 across all 9 models on both DL19 and DL20, and DualEnd methods (Cocktail or Selection) achieve the highest single-method NDCG@10 in 14 of 18 model–dataset configurations (§6.3).
- Standalone BottomUp is consistently and sometimes catastrophically weaker than TopDown. Out of 18 configurations, paired approximate-randomization tests with Bonferroni correction yield **zero** BottomUp wins and **6** significant losses, including a −0.23 NDCG@10 drop on Flan-T5-Large DL19 (§6.2).
- BiDir ensemble does *not* outperform the TopDown baseline. The ensemble inherits BottomUp's noise rather than gaining complementary signal: 0 Bonferroni-significant wins and 3 Bonferroni-significant losses, with the optimal fusion weight α trending toward 1.0 (pure TopDown) (§6.4).
- DualEnd's gains over the strongest TopDown baseline are *directionally consistent* (positive in 14/18 configurations, mean delta +0.0058) but *statistically fragile*: only the Qwen3-4B DL19 gain (+0.0446) survives Bonferroni correction. We therefore frame DualEnd as a quality-first refinement, not a free improvement.
- The mechanism we point to is positional. Standalone worst-selection is dominated by a strong recency bias (the model labels the last passage as "worst" 40–63% of the time across models), while the same model placed in a joint best-and-worst prompt produces a *reversed* primacy-heavy "dual_worst" pattern. Joint elicitation therefore changes the model's behavior, not just its output channel.

The paper's central scientific contribution is therefore not "DualEnd is universally faster or better" but *worst-selection is not the dual of best-selection, and the gap between standalone worst and joint best-worst is mechanistically meaningful.* The most useful practical contribution is the empirical map: which sorting algorithm and which selection direction works for which model family, and where the joint signal is worth its compute.

Contributions. We summarize as follows:

(1) We propose three novel bidirectional strategies for LLM-based setwise ranking that extract more information per LLM call (§3).
(2) We introduce two sorting algorithms—cocktail shaker sort and double-ended selection sort—designed for dual-end LLM comparisons (§4).
(3) We provide formal complexity and information-theoretic analysis comparing all methods (§4).
(4) We conduct comprehensive experiments on TREC DL 2019/2020 with nine LLMs across three architecture families (Flan-T5, Qwen3, and Qwen3.5, 780M–27B), including paired significance testing and a quality–cost Pareto analysis. We complement these with a position-bias study, ranking-agreement analysis, query-difficulty stratification, and a "when DualEnd helps" qualitative summary on representative models (§6, §7).
(5) We outline a refinement package (Selective DualEnd, bias-aware DualEnd, same-call worst-signal regularization) implemented as runnable code paths and motivated by our analyses (§7).
(6) We provide an open-source implementation integrated into the `llm-rankers` framework[1].

2 Related Work

2.1 LLM-Based Document Ranking

LLM-based document ranking has rapidly evolved since the introduction of RankGPT [16], which demonstrated that properly instructed LLMs can deliver competitive results to supervised rerankers. The field has converged on four prompting paradigms:

Pointwise methods [10, 14] score each document independently. MonoT5 [10] uses a sequence-to-sequence model to predict a "true" or "false" relevance label for each query-document pair, using the probability of the "true" token as a relevance score. While highly efficient—each document requires only one inference call and calls are easily parallelizable—pointwise methods suffer from poor ranking effectiveness because they cannot capture relative relevance between documents. A document that is moderately relevant to one query may receive the same score as one that is highly relevant to another, making cross-document comparison unreliable.

[1]Code and run files: https://anonymous

Pairwise methods [12, 13] compare two documents at a time, asking the LLM which is more relevant. PRP [13] achieves competitive effectiveness with GPT-4-class performance using smaller models like Flan-UL2 (20B), outperforming ChatGPT-based pointwise solutions by 4.2% on NDCG@10. The DuoT5 [12] model extends the Expando-Mono-Duo pipeline with a T5-based pairwise reranker. While pairwise methods achieve high effectiveness due to direct inter-document comparison, they require $O(n^2)$ comparisons for all-pairs or $O(n \log n)$ with sorting algorithms—a significant computational overhead.

Listwise methods [9, 16] present multiple documents and request a complete ranking permutation. RankGPT [16] uses a sliding window strategy: a window of w documents slides from the bottom of the initial ranking to the top, with the LLM reranking each window. This approach captures rich inter-document relationships and produces coherent rankings. However, it is constrained by context length, sensitive to input order (position bias), and expensive due to long prompts. Zhang et al. [21] demonstrate that effective listwise rerankers can be built on open-source LLMs, achieving 97% of GPT-4 effectiveness.

Setwise methods [23] ask the LLM to select the most relevant document from a small candidate set (typically 3–4 documents), using the selection as a comparator in sorting algorithms. This paradigm balances effectiveness and efficiency: it extracts more comparative information per call than pairwise (multiple documents compared simultaneously) while using shorter prompts than listwise (only a small subset presented). The approach has been extended by setwise insertion [11], which leverages the initial BM25 ranking as prior knowledge for 31% time reduction; tournament-based ranking (TourRank) [2], which uses sports tournament brackets for more robust rankings; and reasoning-enhanced variants (Rank-R1) [22], which integrate chain-of-thought reasoning via reinforcement learning.

All existing setwise methods share a common design choice: they only ask the LLM to identify the *most relevant* document. Our work is the first to investigate alternative selection targets (least relevant) and simultaneous multi-target selection (both most and least), along with the sorting algorithms that consume those richer outputs.

2.2 Sorting Algorithms for LLM Ranking

Classical sorting algorithms have been adapted as the backbone for pairwise and setwise LLM ranking. The choice of algorithm significantly impacts both the number of LLM calls and the quality of the final ranking:

Heapsort builds an n-ary max-heap where each node is compared against its c children (for setwise with `num_child=` c). Building the heap requires $O(n)$ comparisons, and extracting k maximums requires $O(k \log_c n)$ comparisons, yielding $O(n + k \log_c n)$ total. For the typical setting ($k = 10$, $n = 100$, $c = 3$), this is approximately 130 comparisons.

Bubblesort starts from the bottom of the list and "bubbles" the most relevant document to the top through a series of windowed comparisons. Each of k passes requires $O(n/c)$ comparisons, giving $O(kn/c)$ total—approximately 333 comparisons for the same setting. However, an optimization tracking the last-start position can reduce this significantly by skipping unchanged regions.

Recent work has challenged whether classical complexity bounds remain optimal when the comparator is an LLM. Sato [15] surveys sorting algorithms adapted for LLMs, noting that traditional cost models fail to account for three key properties: (1) LLM comparisons are expensive in both time and cost, (2) comparisons can be batched for parallel execution, and (3) comparisons are noisy and potentially non-transitive. Wisznia et al. [19] further show that batching and caching fundamentally change the cost model, making some "suboptimal" algorithms superior in practice.

Tournament-based approaches like TourRank [2] and BlitzRank [1] represent a move toward extracting richer information from each comparison round. BlitzRank models the complete preference graph from k-wise comparisons rather than discarding loser information. However, these approaches still focus on identifying winners. Our cocktail shaker sort and double-ended selection sort are the first sorting algorithms specifically designed to exploit dual-output (best *and* worst) comparators.

2.3 Position Bias in LLM Ranking

LLMs exhibit systematic bias based on document position in the prompt. The "Lost in the Middle" phenomenon [8] shows that LLMs attend more to information at the beginning and end of long contexts, neglecting middle positions. For ranking, this translates to position-dependent selection probabilities that can distort the final ranking.

Several mitigation strategies have been proposed. Permutation self-consistency [17] shuffles document order across multiple passes and aggregates rankings via voting, achieving 7–18% improvement for GPT-3.5 and 8–16% for LLaMA v2 (70B). However, this multiplies computational cost by the number of permutations. LLM-RankFusion [20] identifies two types of intrinsic inconsistency: *order inconsistency* (conflicting results when document order is swapped) and *transitive inconsistency* (non-transitive preference triads where $A > B$, $B > C$, but $C > A$). These inconsistencies fundamentally challenge sorting-based approaches, which assume a consistent total order.

For setwise ranking specifically, position bias manifests within the small candidate set. Zhuang et al. [23] mitigate this through permutation voting: randomly shuffling both document order and label assignment across multiple permutations, with majority voting to determine the winner.

Our work opens a new dimension of analysis: whether position bias operates differently for best-selection versus worst-selection prompts, and—more interestingly—whether the bias for the *same* model on the *same* candidate set changes when "best" and "worst" are asked jointly versus alone. As we show in §7.1, the dual prompt produces a reversed dual_worst pattern that is consistent across all 9 of our models and gives an empirically grounded mechanism for the directional asymmetry.

2.4 Negative Evidence in LLM Ranking

A growing body of LLM-ranking work also reports negative or surprising findings. Zeng et al. [20] document order- and transitivity-inconsistencies. Tang et al. [17] treat position bias as a defect and

use permutation voting to suppress it. We add to this literature with two coherent negative results: (i) standalone BottomUp is consistently weaker than TopDown, and (ii) directional fusion (BiDir) does not improve over the better individual ranker. These are not isolated tuning failures: they reproduce across 9 models, 2 datasets, multiple fusion methods, and the full range of weight α, and they survive Bonferroni correction in 6/18 and 3/18 configurations respectively (§6.2, §6.4).

3 Methodology

3.1 Preliminaries: Setwise Ranking

Given a query q and a set of n candidate documents $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ from a first-stage retriever, the goal is to produce a ranking π that orders documents by decreasing relevance to q. We focus on the top-k ranking problem, where only the top k positions need to be correctly determined ($k \ll n$).

The setwise approach [23] defines a *comparison function* f_{best} that, given q and a subset $S \subseteq \mathcal{D}$ of size $c + 1$ (where c is the `num_child` parameter), returns the most relevant document:

$$f_{\text{best}}(q, S) = \arg\max_{d \in S} \text{relevance}(q, d) \tag{1}$$

This comparison function serves as the comparator in a sorting algorithm (heapsort or bubblesort) to produce the top-k ranking. Two scoring modes are supported:

- **Generation**: The LLM generates the passage label (e.g., "A"); the decoded output determines the winner.
- **Likelihood**: The logits of candidate passage label tokens are compared directly. For T5 we use direct next-token logits over single-character labels; for Qwen3/Qwen3.5 we score short teacher-forced continuations such as `Passage A`, since causal tokenization does not guarantee that label letters are stable single tokens. Both variants avoid autoregressive decoding.

Prompt template (standard):

```
Given a query "{query}", which of the following
passages is the most relevant one to the query?
Passage A: "{doc_1}"
Passage B: "{doc_2}"
...
Output only the passage label of the most relevant
passage:
```

3.2 Bottom-Up Setwise Ranking

We define a reverse comparison function f_{worst} that selects the *least relevant* document:

$$f_{\text{worst}}(q, S) = \arg\min_{d \in S} \text{relevance}(q, d) \tag{2}$$

Prompt template (reverse):

```
Given a query "{query}", which of the following
passages is the least relevant one to the query?
Passage A: "{doc_1}"
Passage B: "{doc_2}"
...
Output only the passage label of the least relevant
passage:
```

The prompt change is minimal: only two words differ ("least" vs. "most"). This ensures that any observed differences in accuracy or bias are attributable to the cognitive task of identifying the worst versus best document, rather than confounding prompt effects.

Algorithm: We adapt both heapsort and bubblesort for bottom-up selection:

- *Bottom-up heapsort*: We build a *min-heap* where f_{worst} serves as the comparator, placing the least relevant document at the root. We then run a complete heapsort with the worst-selection comparator: each extraction pulls the worst remaining document to the back of the array, and the survivors at the front are guaranteed to be the top-k in their final positions. Crucially, every comparison made during both heap construction and extraction uses the worst-selection prompt.
- *Bottom-up bubblesort*: Instead of bubbling the best document to the top, we sink the worst document to the bottom via forward passes. We run $n - 1$ passes (rather than $n - k$) so that the top-k region is itself sorted by the same worst-selection signal, rather than left in heap order with a stray best-selection clean-up at the end.

Efficiency: Bottom-up heapsort requires $\sim n - 1$ min-extractions (each with $O(\log_c n)$ heapify cost) using only the worst-selection comparator. For the typical IR setting ($k = 10$, $n = 100$), this is $\sim$273 comparisons in our experiments versus $\sim$77 for TopDown-Heap—a $\sim$3.5× increase. Bottom-up bubblesort requires $\sim$1683 comparisons in our setting, more than an order of magnitude over TopDown-Heap. Consequently, BottomUp is more costly for top-k when $k \ll n$, and is primarily valuable for: (i) effectiveness analysis of worst-selection versus best-selection, (ii) as an ensemble component with potentially complementary error patterns, and (iii) scenarios where k is close to n (full ranking).

Hypothesis: We hypothesize that LLMs may exhibit higher accuracy when identifying clearly *irrelevant* documents compared to distinguishing among highly relevant ones. This is because irrelevance detection ("this document about cooking is not relevant to a quantum physics query") often involves categorical topic mismatch that is easy to detect, whereas best-selection among topically relevant candidates requires nuanced assessment of relevance degree. As we will see in §6.2, this hypothesis is decisively refuted on our 9 models.

3.3 Dual-End Setwise Ranking

We define a *dual comparison function* f_{dual} that simultaneously returns both the most and least relevant documents from a candidate set:

$$f_{\text{dual}}(q, S) = \left(\arg\max_{d \in S} \text{rel}(q, d),\ \arg\min_{d \in S} \text{rel}(q, d) \right) \tag{3}$$

Prompt template (dual-end):

```
Given a query "{query}", which of the following
passages is the most relevant and which is the
least relevant to the query?
Passage A: "{doc_1}"
Passage B: "{doc_2}"
...
Output the most relevant passage label and the
least relevant passage label, separated by a comma:
```

Key insight: When selecting the best document from a set, the LLM internally compares all candidates. It implicitly identifies not

only the winner but also the loser—this information is simply never requested. By asking for both, we extract a second ranking decision from the same cognitive process with minimal overhead.

Token overhead analysis: The dual-end prompt adds approximately 15 tokens to the instruction ("*and which is the least relevant to the query*" and the output format specification). The output grows by approximately 1–2 tokens (the second label). For a typical setwise comparison with 4 passages of 128 tokens each, the total prompt is ∼600 tokens. The 15-token overhead is ∼2.5%, while the information gain is ∼74–90% depending on window size (see Table 1).

Scoring modes: Dual-end supports two scoring approaches:

- **Generation mode**: The LLM generates text containing both labels (e.g., "Best: A, Worst: C"). A robust cascading parser handles multiple output formats including "Passage A, Passage C", bracketed labels (`[A]`), numeric labels, and verbose responses. This is the primary mode for Qwen3/Qwen3.5.
- **Likelihood mode**: For T5 we read the full softmax over passage label tokens from a single forward pass and take the maximum as best and minimum as worst. For Qwen3/Qwen3.5 we use the same heuristic but score short continuations like `Passage A` with teacher forcing. We are explicit that this dual-end likelihood path is a best-only-distribution proxy rather than an exact joint $\Pr(\text{Best}{=}X, \text{Worst}{=}Y \mid q, S)$ model. We use it as an efficiency probe and do not over-claim its information-theoretic content.

Output parsing: Reliable extraction of both labels from the generated text is critical. We implement a cascading parser: (1) First, try the explicit "Best: X, Worst: Y" format via regex. (2) If that fails, look for "Passage X" patterns. (3) As a fallback, extract standalone single characters (A–W) from the output. (4) If parsing fails entirely, default to the first and last passage labels and log a warning. The success rate of the primary parser on different models is reported in §6.3.

3.4 Bidirectional Ensemble

We run both TopDown and BottomUp ranking independently on the same candidate set, producing two rankings π_{td} and π_{bu}. These are then fused into a final ranking using rank aggregation.

Motivation: At design time we hypothesized that the error patterns of top-down and bottom-up rankings would be complementary. Top-down errors might occur when the LLM incorrectly identifies a mediocre document as the best in some comparison (false positive at the top), while bottom-up errors might occur when the LLM fails to identify a truly irrelevant document as the worst (false negative at the bottom). Since these error types could be different, fusion might correct errors that either individual method makes. The empirical result (§6.4) is that this complementarity does not materialize because BottomUp's recency-driven errors are too systematic to cancel out, not because the principle of fusion fails.

We evaluate three fusion methods:

- **Reciprocal Rank Fusion (RRF)** [3]: $\text{score}(d) = \sum_{i \in \{\text{td},\text{bu}\}} \frac{1}{k_{\text{rrf}} + \text{rank}_i(d)}$ with $k_{\text{rrf}} = 60$. RRF is widely used for combining heterogeneous ranking signals and is robust to score scale differences.
- **CombSUM**: Normalize ranks to $[0, 1]$ scores per run ($s_i(d) = (n - \text{rank}_i(d) + 1)/n$), then sum. This gives equal weight to both methods.
- **Weighted**: $\text{score}(d) = \alpha \cdot s_{\text{td}}(d) + (1-\alpha) \cdot s_{\text{bu}}(d)$ where α controls the relative weight. We experiment with $\alpha \in \{0.3, 0.5, 0.7, 0.9\}$.

Efficiency: BiDir requires 2× the LLM calls of standard setwise (one full pass each for top-down and bottom-up). This is comparable to permutation voting with $p = 2$ [23], which also doubles the cost but provides order-based rather than direction-based complementary information. A key experimental question is which form of complementarity (direction vs. order) yields greater improvement per unit of additional cost.

4 Sorting Algorithms for Dual-End Comparisons

Standard heapsort and bubblesort are designed for single-output comparators that return one winner per call. To fully leverage f_{dual}, which returns *two* outputs per call, we need algorithms that can exploit both the best and worst information simultaneously. We introduce two such algorithms and then describe a refinement package built on top of them.

4.1 Cocktail Shaker Sort

Cocktail shaker sort (also known as bidirectional bubble sort or cocktail sort) is a well-known variant of bubble sort in classical computer science [6]. We adapt it for LLM-based ranking with dual-end comparisons.

The key idea is to alternate between two types of passes:

(1) **Backward pass** (bottom → top): A sliding window moves from the bottom of the unsorted region toward the top. In each window, f_{dual} identifies both the best and worst documents. The best is moved to the front of the window (bubbling up), while the worst is moved to the back (sinking down).
(2) **Forward pass** (top → bottom): The window slides in the opposite direction, from the top of the unsorted region toward the bottom. Again, f_{dual} identifies both extremes, with the worst sinking down as the primary operation and the best bubbling up as a bonus.

Swap handling: When both the best and worst need to be repositioned within the same window, care must be taken to avoid interference. We handle three cases: (1) if best is at the back and worst is at the front, a single swap suffices; (2) otherwise, move the primary element first (best for backward passes, worst for forward passes), then adjust the secondary element's index if it was displaced by the first swap.

Complexity: Each backward pass consists of $\lceil (b-t)/c \rceil$ dual comparisons, and each forward pass is similar. In each iteration of the while loop, the sorted region grows by 1 at the top (from the backward pass) and potentially by 1 at the bottom (from the forward pass sinking the worst). Thus, each iteration sorts up to 2 elements using roughly $2 \cdot \lceil (b-t)/c \rceil$ dual comparisons. For top-k ranking, we need at most k backward passes (the forward passes are bonus), giving $O(kn/c)$ dual comparisons—the same asymptotic count as standard bubblesort, but each comparison also produces bottom-ranking information.

Algorithm 1 Cocktail Shaker Sort for Dual-End Setwise

Require: Documents $D[0..n-1]$, query q, top-k, window size $c+1$
$t \leftarrow 0$ {Top sorted boundary}
$b \leftarrow n$ {Bottom unsorted boundary}
while $t < k$ **do**
  // **Backward pass: bubble best to position** t
  for window W from position $b-c-1$ down to t **do**
    $(best, worst) \leftarrow f_{\text{dual}}(q, W)$
    Move $best$ to front of W; move $worst$ to back of W
  end for
  $t \leftarrow t + 1$; $b \leftarrow b - 1$
  if $t < k$ **then**
    // **Forward pass: sink worst to position** $b-1$
    for window W from position t up to $b-1$ **do**
      $(best, worst) \leftarrow f_{\text{dual}}(q, W)$
      Move $worst$ to back of W; move $best$ to front of W
    end for
    $b \leftarrow b - 1$
  end if
end while
return $D[0..k-1]$

4.2 Double-Ended Selection Sort

This algorithm takes a more aggressive approach: in each round, it finds the *global* best and worst among all unsorted documents, placing one at the next top position and the other at the next bottom position.

Algorithm 2 Double-Ended Selection Sort

Require: Documents $D[0..n-1]$, query q, top-k, group size $c+1$
$t \leftarrow 0$; $b \leftarrow n - 1$
while $t < k \wedge t < b$ **do**
  // **Phase 1: Group comparisons**
  Divide $D[t..b]$ into groups of size $\leq c+1$
  for each group G **do**
    $(best_G, worst_G) \leftarrow f_{\text{dual}}(q, G)$
  end for
  // **Phase 2: Winner and loser tournaments**
  $d^* \leftarrow$ tournament among group $best_G$'s using f_{best}
  $d_\perp \leftarrow$ tournament among group $worst_G$'s using f_{worst}
  Place d^* at position t; place $d_\perp$ at position b
  $t \leftarrow t + 1$; $b \leftarrow b - 1$
end while
return $D[0..k-1]$

Each round has two phases. In Phase 1, the unsorted documents are divided into groups of size $c + 1$, and f_{dual} is called on each group to identify local winners and losers. In Phase 2, the group winners compete in a secondary tournament using f_{best} to find the global best, and group losers compete using f_{worst} to find the global worst. Both are placed at their respective positions, and the unsorted region shrinks by 2.

Complexity: Let $n' = b - t + 1$ be the current unsorted size. Phase 1 requires $\lceil n'/(c+1) \rceil$ dual comparisons. Phase 2 requires at most $2 \cdot \lceil g/(c+1) \rceil$ single comparisons where $g = \lceil n'/(c+1) \rceil$ is the number of groups. Each round places 2 elements, so $\lceil k/2 \rceil$ rounds suffice. For $k = 10$, $n = 100$, $c = 3$: roughly ~25 group dual-comparisons + ~14 single tournament comparisons per round, ~195 comparisons total over 5 rounds. This exceeds standard heapsort (~130) but extracts richer information per round and naturally produces bottom-ranking as a byproduct.

4.3 Refinement Variants

The Pareto analysis in §7.5 shows that the global mean comparison- and token-frontier in our setting contains only TopDown-Heap, TopDown-Bubble, and DualEnd-Cocktail. TopDown-Heap is the cheap anchor, TopDown-Bubble is the strongest middle-cost anchor, and DualEnd-Cocktail is the quality-first anchor. This identifies a clear refinement target: a method that sits *between* TopDown-Bubble and DualEnd-Cocktail. We implement three such refinements as runnable code paths in the `llm-rankers` fork.

Selective DualEnd. A TopDown sort that upgrades to same-call best-worst prompting only on routed windows, and otherwise issues a standard single-output prompt. We support three gating strategies: (i) shortlist routing (only windows that overlap the current top-k shortlist), (ii) query-locally uncertain routing (only windows whose BM25 score spread exceeds a query-local percentile threshold over sliding windows in the original ranking), and (iii) the union of the two. For the heapsort variant we disable shortlist routing because heap nodes are not stable rank positions, and use uncertainty routing only.

Bias-aware DualEnd. An order-robust DualEnd variant motivated directly by the reversed dual_worst pattern (§7.1). On gated hard windows, we run a small set of controlled orderings (base, reversed, and a label-shifted variant) and majority-vote the dual labels back into the original window order. Off the gated path, we run a single ordering. We restrict bias-aware DualEnd to the bubblesort/cocktail and selection drivers because the heap traversal does not exercise the order-robust joint prompt path.

Same-call worst-signal regularization. A head-focused TopDown pass that uses the same-call worst output only as a local negative constraint. We promote best as usual; we only push worst locally to the back when the candidate is already outside the protected ranking head frontier (top-k plus one active window). This avoids full backward DualEnd passes while still consuming the worst signal that the dual prompt produces "for free" in the same call.

These three variants share the same gating infrastructure (query-local BM25 spread percentiles, configurable shortlist overlap) and expose method-level counters (dual invocations, single invocations, order-robust windows, extra orderings, regularized worst moves) so they can be audited as method behavior rather than as black-box NDCG numbers. We use them in this paper as motivated by the analysis in §7 but reserve the full empirical evaluation of these variants for follow-up work, since the paper's headline question is about asymmetry rather than about routing strategy tuning.

4.4 Information-Theoretic Analysis

We quantify the "information efficiency" of each comparison strategy by computing the Shannon entropy of the ranking information extracted per LLM call. For a uniform prior over m candidates:

- **Best-only** (f_{best}): Identifying 1 of m candidates yields $\log_2(m)$ bits.
- **Worst-only** (f_{worst}): Same as best-only: $\log_2(m)$ bits.
- **Dual-end** (f_{dual}): Identifying the best from m, *then* the worst from the remaining $m-1$, yields $\log_2(m)+\log_2(m-1) = \log_2\big(m(m-1)\big)$ bits.
- **Full ranking** (listwise): Identifying a complete ranking of m candidates yields $\log_2(m!)$ bits.

Table 1 shows the information per call and per token for different window sizes. The key observation is that dual-end achieves ∼74–90% more information per call at only ∼2.5% more tokens, yielding dramatically higher *information per token*.

Table 1: Information extracted per LLM call for different comparison strategies. Token overhead calculated for passages of 128 tokens.

Strategy	Bits/call	ΔTokens	Bits/token
Window size $m = 4$ ($c = 3$)			
Best-only	2.00	0	baseline
Worst-only	2.00	0	baseline
Dual-end	3.58	+25	+74%
Window size $m = 6$ ($c = 5$)			
Best-only	2.58	0	baseline
Dual-end	4.91	+25	+87%
Window size $m = 8$ ($c = 7$)			
Best-only	3.00	0	baseline
Dual-end	5.81	+25	+90%

This analysis assumes the LLM's selections are accurate. In practice, the LLM may have different accuracy for best-selection versus worst-selection, which affects the *effective* information per call. As we will see, standalone worst-selection is dominated by recency bias, while joint best-worst elicitation produces a different positional pattern entirely. We measure the empirical effect on ranking quality in §6.3 and on bias structure in §7.1.

5 Experimental Setup

5.1 Datasets and Metrics

We evaluate on two groups of benchmarks:

Primary benchmarks (TREC Deep Learning):

- **TREC DL 2019** [5]: 43 queries with graded relevance judgments (0–3) over the MS MARCO passage corpus (8.8M passages).
- **TREC DL 2020** [4]: 54 queries with the same passage corpus and grading scheme.

Generalizability benchmarks (BEIR, in progress): To assess domain generalizability, we additionally run a representative subset of BEIR [18] datasets on three models (Flan-T5-XL, Qwen3-8B, Qwen3.5-9B). These runs are still completing on a remote cluster and will be folded into the camera-ready version. The TREC DL 2019/2020 results below are the basis of all of our headline claims.

For all datasets, we rerank the top-100 BM25 results from Pyserini [7] using default parameters.

Effectiveness metrics: NDCG@10 is our primary metric, following prior work on LLM-based reranking [13, 16, 23]. We also report MAP@100 for completeness in the main results table.

Efficiency metrics: We report (i) the number of LLM comparison calls per query (#Comps), (ii) total prompt tokens per query, (iii) total completion tokens per query, and (iv) wall-clock time per query.

Statistical testing: We compare each challenger family (Dual-End, BottomUp, BiDir) against the best TopDown baseline for the same model-dataset configuration using two-sided paired approximate randomization with 100,000 samples. Mean-delta uncertainty is reported as a paired bootstrap 95% confidence interval (20,000 resamples). Multiple testing is controlled by Bonferroni correction within each challenger family across the 18 TREC DL configurations. Per-query `ndcg_cut_10` values are read directly from the saved `.eval` files in the experiment output tree, so the test statistic is grounded in the same numbers as the main results table. Full per-config tables and verdicts appear in the supplementary `SIGNIFICANCE_TESTS` artifact and are summarized in §6.3.

5.2 Models

We select models from three architecture families spanning a range of sizes to assess generalizability:

Encoder-decoder (Flan-T5): Instruction-tuned sequence-to-sequence models that natively support both generation and likelihood scoring modes via direct logits over single-character labels.

- **Flan-T5-Large** (780M parameters)
- **Flan-T5-XL** (3B parameters) — used as the primary model for the bulk of ablations.
- **Flan-T5-XXL** (11B parameters) — for scaling analysis.

Decoder-only (Qwen3): Instruction-tuned causal language models with chain-of-thought reasoning. These models emit `<think>...</think>` reasoning blocks, which we strip during output parsing. The main paper tables use generation mode; the codebase also supports a likelihood follow-up that scores short teacher-forced answer strings such as `Passage A` without decoding.

- **Qwen3-4B** (4B parameters)
- **Qwen3-8B** (8B parameters)
- **Qwen3-14B** (14B parameters)

Hybrid causal (Qwen3.5): A novel architecture combining Gated DeltaNet linear attention (75% of layers) with standard softmax attention (25%), resulting in a decoder-only model with subquadratic complexity. Despite the `ForConditionalGeneration` class name (which refers to its multimodal capability), these are causal language models loaded via `AutoModelForCausalLM`. The main paper tables use generation mode; the codebase also supports the same teacher-forced likelihood follow-up used for Qwen3.

- **Qwen3.5-4B** (4B parameters)
- **Qwen3.5-9B** (9B parameters)
- **Qwen3.5-27B** (27B parameters)

All models are run in half precision (FP16/BF16) on a single NVIDIA H100 80GB GPU. Flan-T5 models use passage truncation length of 128 tokens (following Zhuang et al. [23], constrained by 512-token model context). Qwen3/3.5 models use 512 tokens

(32k+ context window). We study the effect of passage length in an ablation (§6.5).

5.3 Configurations

Table 2 summarizes all experimental configurations. For all setwise methods, we set `num_child= 3` (4-way comparison) following Zhuang et al. [23]. We rerank the top 100 BM25 results and extract the top $k = 10$. Passage truncation length varies by model family: 128 tokens for Flan-T5 and 512 tokens for Qwen3/3.5.

Table 2: Experimental configurations. All use k=10, hits = 100, c=3.

Method	Direction	Algorithm	Extra
TopDown-Heap	topdown	heapsort	—
TopDown-Bubble	topdown	bubblesort	—
BottomUp-Heap	bottomup	heapsort	—
BottomUp-Bubble	bottomup	bubblesort	—
DualEnd-Cocktail	dualend	bubblesort	cocktail shaker
DualEnd-Selection	dualend	selection	DE selection
BiDir-RRF	bidirectional	heapsort	RRF (k_{rrf}=60)
BiDir-Weighted	bidirectional	heapsort	α=0.7

Implementation: All methods are implemented in the `llm-rankers` framework [23], extending the existing `SetwiseLlmRanker` class. The `BottomUpSetwiseLlmRanker`, `DualEndSetwiseLlmRanker`, and `BidirectionalEnsembleRanker` classes are added as new modules, alongside the three refinement variants `SelectiveDualEndSetwiseLlmRanker`, `BiasAwareDualEndSetwiseLlmRanker`, and `SameCallRegularizedSetwiseLlmRanker` described in §4.3. The bidirectional ensemble uses Python's `__new__` constructor pattern to share model weights between the top-down and bottom-up rankers, avoiding redundant GPU memory usage. For Qwen3-family thinking models, output parsing handles `<think>...</think>` reasoning blocks via regex stripping, with `max_new_tokens` = 256 to allow the thinking block to complete before the answer (and 512 for the longer dual-output prompt). The causal likelihood follow-up scores short continuations such as `Passage A` with teacher forcing rather than assuming a single stable label token.

6 Results

We organize our results around three research questions:

- **RQ1**: Does bottom-up (worst) selection achieve different effectiveness than top-down (best) selection?
- **RQ2**: Does dual-end selection improve effectiveness, and at what cost?
- **RQ3**: Does bidirectional ensemble improve over individual methods?

6.1 Main Results

Table 3 presents the main results across all configurations, models, and datasets.

High-level pattern. The DualEnd family is the strongest single-method family overall: DualEnd-Cocktail wins 11 of 18 configurations, DualEnd-Selection wins another 3 (Qwen3-4B on both DL19 and DL20 plus Qwen3-8B DL19), and TopDown-Bubble wins the remaining 4. TopDown-Heap, BottomUp-Heap, BottomUp-Bubble, BiDir-RRF, and BiDir-Weighted never win the headline NDCG@10 cell at the 4-decimal level. Mean NDCG@10 across the 9 models is DualEnd-Cocktail ≥ DualEnd-Selection ≥ TopDown-Bubble ≥ TopDown-Heap on both DL19 and DL20, with all four families well separated from BottomUp and BiDir.

6.2 RQ1: Bottom-Up vs. Top-Down Effectiveness

We examine whether LLMs achieve different ranking effectiveness when asked to identify the *least* relevant document compared to the *most* relevant.

Overall effectiveness: Table 3 reveals a consistent pattern: BottomUp methods are substantially weaker than their TopDown counterparts across all models. On DL 2019 with Flan-T5-XL, TopDown-Heap achieves .6901 NDCG@10 while BottomUp-Heap achieves .6630—a gap of 2.7 points. On DL 2020, the gap widens to 4.4 points (.6680 vs. .6241).

The gap is dramatically *larger* for smaller models: with Flan-T5-Large, BottomUp-Heap drops to .2888 on DL 2019 (vs. .6541 for TopDown)—a catastrophic 36.5-point gap. For larger models, the gap narrows: with Qwen3-14B, BottomUp-Heap achieves .6966 (vs. .7447 for TopDown, a 4.8-point gap) and with Qwen3.5-27B, the gap is 3.1 points (.7135 vs. .7449). This scale-dependence suggests that worst-selection requires more model capacity than best-selection.

Bubblesort vs. heapsort: Within the BottomUp family, bubblesort outperforms heapsort for some larger models: on DL 2019 with Qwen3.5-27B, BottomUp-Bubble achieves .7336 versus .7135 for BottomUp-Heap. However, for smaller models the pattern reverses (T5-Large: .4571 vs. .2888). Bubblesort's $n - 1$ forward passes provide more opportunities for error correction through repeated comparisons of neighboring elements, but at ∼1683 comparisons per query it is more than twenty times the cost of TopDown-Heap.

Significance. Paired approximate randomization with Bonferroni correction over the 18 configurations confirms that BottomUp's loss against the best TopDown baseline is statistically robust. The mean delta is −0.0616, the family is positive in 0/18 configurations, and 6 configurations exceed the corrected significance threshold: Flan-T5-Large DL19 (−0.2302, $p_{\text{Bonf}} < 0.001$), Flan-T5-Large DL20 (−0.2148, $p_{\text{Bonf}} < 0.001$), Qwen3-14B DL20 (−0.0649, $p_{\text{Bonf}} = 0.001$), Qwen3.5-4B DL19 (−0.0950, $p_{\text{Bonf}} = 0.005$), Qwen3.5-4B DL20 (−0.0749, $p_{\text{Bonf}} = 0.016$), and Qwen3.5-9B DL19 (−0.0570, $p_{\text{Bonf}} = 0.025$). No BottomUp variant produces a Bonferroni-significant win on any TREC DL configuration.

Refuting the irrelevance-detection hypothesis: Our initial hypothesis—that LLMs may be better at identifying clearly irrelevant documents—is decisively refuted on these 9 models. Worst-selection is harder than best-selection across all model sizes, and the difficulty gap is largest for the smallest models. We conjecture that "least relevant" requires the model to hold a coherent relevance ordering of all candidates and select the minimum, while "most relevant" often has a salient winner that stands out. The position-bias analysis in §7.1 provides additional evidence: standalone worst-selection collapses heavily onto the last passage (40–63% of the time across models), suggesting that the model defaults to a positional heuristic rather than a relevance judgment.

Table 3: Main results on TREC DL 2019 and 2020 (NDCG@10 and MAP@100). Best NDCG@10 result per model in bold, second-best underlined. All use `num_child`=3, k=10, hits=100. The full 8-method block is shown for every model.

Method	TREC DL 2019		TREC DL 2020	
	NDCG@10	MAP@100	NDCG@10	MAP@100
Flan-T5-Large (780M), generation, PL=128				
TopDown-Heap	.6541	.3074	.6100	.3511
TopDown-Bubble	**.6874**	.3087	.6264	.3482
BottomUp-Heap	.2888	.1737	.2203	.1786
BottomUp-Bubble	.4571	.2374	.4116	.2433
DualEnd-Cocktail	.6708	**.3136**	**.6308**	**.3553**
DualEnd-Selection	.6420	.3016	.6081	.3350
BiDir-RRF	.5820	.2855	.5580	.3251
BiDir-Weighted	.6147	.3062	.5805	.3371
Flan-T5-XL (3B), generation, PL=128				
TopDown-Heap	.6901	.3172	.6680	.3696
TopDown-Bubble	**.6980**	.3182	**.6868**	.3773
BottomUp-Heap	.6630	.3060	.6241	.3511
BottomUp-Bubble	.6730	.2995	.6691	.3638
DualEnd-Cocktail	.6884	.3095	.6795	**.3798**
DualEnd-Selection	.6792	.3057	.6600	.3750
BiDir-RRF	.6845	.3215	.6509	.3703
BiDir-Weighted	.6810	.3176	.6528	.3673
Flan-T5-XXL (11B), generation, PL=128				
TopDown-Heap	.6846	.3201	.6790	.3727
TopDown-Bubble	.7077	.3114	**.6959**	.3693
BottomUp-Heap	.6874	.3211	.6625	.3625
BottomUp-Bubble	.6936	.3114	.6811	.3681
DualEnd-Cocktail	**.7137**	.3155	.6895	.3718
DualEnd-Selection	.6974	.3155	.6754	.3787
BiDir-RRF	.6905	**.3265**	.6673	.3726
BiDir-Weighted	.6734	.3215	.6649	.3694
Qwen3-4B (4B), generation, PL=512				
TopDown-Heap	.6775	.3115	.6488	.3479
TopDown-Bubble	.6491	.2904	.6269	.3288
BottomUp-Heap	.6261	.2916	.5963	.3236
BottomUp-Bubble	.6305	.2765	.5778	.3060
DualEnd-Cocktail	.6796	.3064	.6454	.3436
DualEnd-Selection	**.7220**	**.3265**	**.6627**	**.3484**
BiDir-RRF	.6814	.3369	.6245	.3504
BiDir-Weighted	.6608	.3072	.6376	.3479
Qwen3-8B (8B), generation, PL=512				
TopDown-Heap	.6819	.3241	.6532	.3591
TopDown-Bubble	.6794	.3034	.6372	.3433
BottomUp-Heap	.6431	.2897	.5963	.3262
BottomUp-Bubble	.6273	.2857	.5948	.3123
DualEnd-Cocktail	.7155	.3239	**.6678**	.3653
DualEnd-Selection	**.7158**	.3256	.6583	.3607
BiDir-RRF	.6826	.3225	.6600	**.3765**
BiDir-Weighted	.6784	.3197	.6397	.3537
Qwen3-14B (14B), generation, PL=512				
TopDown-Heap	.7447	.3420	.6962	.3783
TopDown-Bubble	.7455	.3354	.7044	.3875
BottomUp-Heap	.6966	.3244	.6323	.3463
BottomUp-Bubble	.6702	.3027	.6395	.3491
DualEnd-Cocktail	**.7519**	.3403	**.7051**	**.3907**
DualEnd-Selection	.7475	**.3457**	.7003	.3863
BiDir-RRF	.7172	.3431	.6736	.3775
BiDir-Weighted	.7200	.3423	.6741	.3746
Qwen3.5-4B (4B), generation, PL=512				
TopDown-Heap	.7087	.3289	.6521	.3678
TopDown-Bubble	.7108	.3258	.6713	.3755
BottomUp-Heap	.6158	.2881	.5550	.3076
BottomUp-Bubble	.6120	.2850	.5963	.3263
DualEnd-Cocktail	**.7161**	.3210	**.6768**	**.3773**
DualEnd-Selection	.7022	**.3288**	.6487	.3471
BiDir-RRF	.6614	.3167	.6403	.3547
BiDir-Weighted	.6714	.3204	.6567	.3638
Qwen3.5-9B (9B), generation, PL=512				
TopDown-Heap	.7329	.3418	.6950	.3870
TopDown-Bubble	.7349	**.3415**	.6925	.3847
BottomUp-Heap	.6779	.3194	.6319	.3470
BottomUp-Bubble	.6712	.3051	.6677	.3700
DualEnd-Cocktail	**.7370**	.3356	**.6984**	.3920
DualEnd-Selection	.7309	.3399	.6927	**.3942**
BiDir-RRF	.7101	.3347	.6647	.3722
BiDir-Weighted	.7087	.3349	.6768	.3798
Qwen3.5-27B (27B), generation, PL=512				
TopDown-Heap	.7449	**.3521**	.7104	.4010
TopDown-Bubble	.7435	.3481	.7178	.4036
BottomUp-Heap	.7135	.3357	.6426	.3786
BottomUp-Bubble	.7336	.3378	.7004	.3971
DualEnd-Cocktail	**.7475**	.3492	**.7186**	**.4075**
DualEnd-Selection	.7319	.3456	.7122	.4036
BiDir-RRF	.7198	.3457	.6781	.3980
BiDir-Weighted	.7229	.3458	.6871	.3995

6.3 RQ2: Dual-End Effectiveness and Efficiency

We assess whether DualEnd can improve effectiveness, and at what cost.

Parsing success rate: Reliable dual output parsing is critical. For Flan-T5 models, we use likelihood scoring internally for the dual-end comparison—reading both the maximum and minimum of the softmax distribution over passage labels in a single forward pass. This avoids output parsing entirely and achieves 100% success rate. For Qwen3/3.5 models, a cascading parser handles varied output formats including bracketed labels ([A]), numeric labels, and verbose responses. The overall parse failure rate (requiring heuristic defaults) is below 1% across all models. The worst case is Qwen3-14B on DL 2020 with a 0.9% partial parse rate (254 out of ~29K comparisons).

Effectiveness: The DualEnd family is the strongest overall. DualEnd-Cocktail ranks first in mean NDCG@10 across all 9 models on both DL 2019 and DL 2020, and it is the strongest *single* method in 11 of 18 model–dataset configurations in Table 3. DualEnd-Selection wins in 3 additional configurations (Qwen3-4B DL19, Qwen3-4B DL20, Qwen3-8B DL19), so DualEnd methods jointly achieve the highest NDCG@10 in 14 of 18 configurations. On DL 2019, representative gains include .7137 with T5-XXL (vs. .7077 for TopDown-Bubble), .7519 with Qwen3-14B (vs. .7455), and .7475 with Qwen3.5-27B (vs. .7449). The gain over the best top-down baseline ranges from +0.0026 to +0.0446 NDCG@10 points depending on model and dataset, with one exception (DualEnd-Cocktail vs. TopDown-Bubble on Flan-T5-Large DL19 is −0.0165).

DualEnd-Selection is especially competitive on smaller Qwen models, achieving .7220 with Qwen3-4B on DL 2019 and .6627 on DL 2020, where it outperforms all other methods by margins of +0.0445 and +0.0139 respectively. More broadly, the selection sort variant tends to favor smaller Qwen models, while cocktail shaker sort is more robust across model scales and families.

Significance. Across the 18 TREC DL configurations, the DualEnd family is positive against the best TopDown baseline in 14 cases (mean delta +0.0058). However, only one configuration survives Bonferroni correction: Qwen3-4B DL 2019, where DualEnd-Selection beats TopDown-Heap by +0.0446 ($p_{\text{Bonf}} = 0.010$, 95% CI [+0.0216, +0.0695], 28-13-2 wins-losses-ties). A near-miss is Qwen3-8B DL 2019 with +0.0340 ($p_{\text{Bonf}} = 0.727$, 26-12-5). The remaining 16 configurations are consistent with the directional pattern but their bootstrap intervals cross zero, so we interpret DualEnd as the strongest empirical family overall while not claiming a uniform statistically significant improvement on these 43/54-query test sets.

Why does DualEnd work? The cocktail shaker sort alternates between forward passes (sinking the worst) and backward passes (bubbling the best). Each comparison identifies both extremes, so each pass makes progress at *both* ends of the ranking. This dual progress is the key advantage: while TopDown-Bubble only moves the best document per pass, DualEnd-Cocktail simultaneously moves the worst to the bottom—effectively getting two sorting operations per LLM call. The position-bias analysis in §7.1 adds a second mechanism: the dual prompt restructures the model's positional defaults, with DualEnd's "best" selection becoming more uniform and its "worst" selection developing a reversed primacy pattern relative to standalone BottomUp.

Efficiency: The efficiency story of DualEnd is nuanced. Table 4 compares the methods.

Table 4: Efficiency comparison for top-k=10 ranking of n=100 documents with c=3. Theoretical complexity and empirical measurements (per query, averaged across DL 2019).

Method	Theory	Comps	Tokens	Time
TopDown-Heap	$O(n+k\log_c n)$	77	32K	7s
TopDown-Bubble	$O(kn/c)$	314	129K	27s
BottomUp-Heap	$O(n\log_c n)$	273	112K	23s
BottomUp-Bubble	$O((n-1)n/c)$	1683	691K	146s
DualEnd-Cocktail	$O(kn/c)$	546	224K	30s
DualEnd-Selection	$O(\frac{k}{2}\cdot\frac{n}{c+1})$	406	159K	27s
BiDir-RRF	2× Heap	350	144K	30s

DualEnd-Cocktail uses 546 comparisons—about 7× TopDown-Heap's 77 and 1.7× TopDown-Bubble's 314—but achieves higher mean NDCG@10. The additional comparisons are not wasted: each dual comparison produces *two* ranking decisions, and the cocktail shaker sort exploits both to make progress from both ends of the ranking. DualEnd-Selection uses 406 comparisons (29% fewer than cocktail) with competitive effectiveness, especially on the smallest Qwen models.

The token-level overhead is similar: DualEnd-Cocktail uses 224K total tokens per query versus 129K for TopDown-Bubble, a 74% increase. We therefore frame DualEnd explicitly as a quality-first refinement rather than a free improvement: the headline contribution is "more bits per call extracted in a structurally useful way," not "fewer total calls." The Pareto analysis in §7.5 formalizes this and identifies a missing intermediate point between TopDown-Bubble and DualEnd-Cocktail that motivates the refinement variants in §4.3.

6.4 RQ3: Bidirectional Ensemble

We evaluate whether fusing top-down and bottom-up rankings improves over either individual method.

Ensemble effectiveness: Contrary to our hypothesis, BiDir does *not* outperform the TopDown baseline. On DL 2019 with Flan-T5-XL, BiDir-RRF achieves .6845 and BiDir-Weighted achieves .6810—both *below* TopDown-Heap (.6901). This pattern holds across all 9 models and both datasets. The best BiDir result on DL 2019 is .7229 with Qwen3.5-27B, still below TopDown-Heap's .7449. Per-query analysis reveals that fusion "beats the best individual" on only 8–11 queries while it lands "worse than the worst individual" on 10–11 queries—a net negative effect.

Significance. The mean BiDir delta against the best TopDown baseline is −0.0232. The family is positive in 3/18 configurations (Qwen3-4B DL19 +0.0039, Qwen3-8B DL19 +0.0008, Qwen3-8B DL20 +0.0068), but none of these are Bonferroni-significant. By contrast, BiDir produces 3 Bonferroni-significant losses: Flan-T5-Large DL19 (−0.0727, $p_{\text{Bonf}} = 0.013$), Flan-T5-Large DL20 (−0.0459, $p_{\text{Bonf}} = 0.035$), and Qwen3.5-27B DL20 (−0.0307, $p_{\text{Bonf}} = 0.016$). The asymmetry between (no significant wins) and (multiple significant losses) is the strongest evidence that direction-based ensembling does not work in this regime.

Why does BiDir fail to improve? The weakness of the BottomUp component drags down the ensemble. Since BottomUp-Heap is consistently 2.7–36.5 points below TopDown-Heap, fusing these two rankings dilutes the stronger signal. Even with $\alpha = 0.7$ (giving 70% weight to the top-down component), the bottom-up noise cannot be fully suppressed. Ranking agreement analysis (§7.3) confirms: TopDown and DualEnd share Kendall's $\tau \approx 0.93$ (high agreement), while TopDown and BottomUp only reach $\tau \approx 0.86$—the divergence is not complementary, it is noise.

Fusion weight sensitivity: We test $\alpha \in \{0.3, 0.5, 0.7, 0.9\}$ (Table 7). Consistently across all models, $\alpha = 0.9$ (90% TopDown, 10% BottomUp) is optimal. The monotonic improvement with increasing α confirms that BottomUp contributes only noise; the limit $\alpha \longrightarrow 1$ is just plain TopDown.

Comparison with permutation voting: We compare BiDir (2× cost) with permutation voting ($p = 2$, also 2× cost). Permutation voting uses the same 2× budget but applies it to the same algorithm with different orderings, addressing position bias rather than directional complementarity. On Qwen3-14B DL 2019, BiDir-RRF (.7172) loses to TopDown-Heap (.7447), while DualEnd-Cocktail beats BiDir-RRF on 23 of 43 queries at lower cost. This reinforces the conclusion that, when extra compute is available, redirecting it into joint best-worst elicitation (DualEnd) is more useful than redirecting it into a worst-only ranking pass (BiDir).

Negative result significance: This negative result challenges the intuitive assumption that combining rankings from different "perspectives" will always help. The key difference from permutation voting [23] is that permutation voting combines rankings from the *same* algorithm with different input orders (addressing position bias), while BiDir combines rankings from *different* algorithms where one is fundamentally weaker. Directional complementarity is not analogous to order-based complementarity.

6.5 Ablation Studies

Effect of window size (`num_child`): Table 5 shows the effect of varying $c \in \{2, 3, 5, 7\}$ on DualEnd-Cocktail effectiveness on DL 2019.

Table 5: Effect of `num_child` (c) on DualEnd-Cocktail (DL 2019). Optimal c depends on model family.

c	Window	T5-XL	Qwen3-8B	Q3.5-9B	#Comps
2	3-way	.6988	.7187	.7392	815
3	4-way	.6884	.7155	.7370	546
5	6-way	.6749	.7224	.7336	333
7	8-way	.6480	.7249	.7386	247

The effect of window size is model-family-dependent. For Flan-T5-XL (512-token context limit), smaller windows are better: $c = 2$ (.6988) significantly outperforms $c = 7$ (.6480), a 5.1-point gap. Larger windows cause more prompt truncation, degrading both best and worst selection accuracy. For Qwen3-8B (32K+ context), larger windows are slightly better: $c = 7$ (.7249) edges $c = 2$ (.7187). For Qwen3.5-9B, performance is stable across window sizes (.7336–.7392). Importantly, larger windows reduce comparison count (247 at $c = 7$ vs. 815 at $c = 2$), offering a quality-efficiency trade-off.

Effect of passage length: Table 6 reports the effect of passage truncation length on both TopDown-Heap and DualEnd-Cocktail for one T5 model and two Qwen models.

Table 6: Effect of passage truncation length on NDCG@10 (DL 2019). T5-XL has a 512-token context limit; Qwen3-8B and Qwen3.5-9B have 32K+.

PL	T5-XL		Qwen3-8B		Q3.5-9B	
	TD-H	DE-C	TD-H	DE-C	TD-H	DE-C
64	.6821	.6846	.6772	.7035	.7207	.7142
128	.6901	.6884	.6934	.7168	.7378	.7406
256	.6890	.6918	.6950	.7130	.7329	.7393
512	.6890	.6879	.6895	.7122	.7329	.7370

For Flan-T5-XL, performance plateaus at PL=128 (the original paper's default), with PL=256 and PL=512 producing nearly identical results—the model's 512-token context limit means longer passages are truncated. For Qwen models, PL=128 provides a small boost over PL=64 (+1.3–1.7 points), but PL=256 and PL=512 offer negligible further improvement. Crucially, the *relative* advantage of DualEnd-Cocktail over TopDown-Heap is consistent across all passage lengths, confirming our findings are not artifacts of a particular truncation setting.

Fusion weight sensitivity (BiDir-Weighted): For BiDir-Weighted, we vary $\alpha \in \{0.3, 0.5, 0.7, 0.9\}$ to control the relative importance of top-down versus bottom-up. Across all three representative models, $\alpha = 0.9$ (90% TopDown weight) is consistently optimal, confirming that the BottomUp component adds noise rather than complementary signal.

Table 7: Effect of fusion weight α for BiDir-Weighted on DL 2019 (NDCG@10). α=1.0 is pure TopDown; α=0.0 is pure BottomUp.

α	0.3	0.5	0.7	0.9
T5-XL	.6814	.6845	.6810	.7002
Q3-8B	.6618	.6638	.6784	.6898
Q3.5-9B	.6984	.7087	.7087	.7240

7 Analysis and Discussion

We now look beyond aggregate NDCG@10 numbers and ask *why* the directional pattern in §6 arises. Four lenses inform this: position bias (§7.1), query difficulty (§7.2), ranking agreement (§7.3), and a quality-cost Pareto view that frames the refinement target (§7.5).

7.1 Position Bias Analysis

We investigate a question unique to our framework: do LLMs exhibit different position bias patterns when asked to select the worst versus the best document, and—more importantly—does the same model behave differently when "best" and "worst" are asked *jointly* versus alone?

Experimental design: For each comparison, we record which passage label (A, B, C, or D) was selected, along with the comparison type (`best`, `worst`, `dual_best`, `dual_worst`). We analyze approximately 170K comparisons per model across all methods.

Best-selection bias: For f_{best} (TopDown), we observe a strong U-shaped bias across all models: positions A (first) and D (last) are consistently overselected, while positions B and C are underselected. For Qwen3-14B on DL 2019 (representative): $A = 35.1\%$, $B = 12.5\%$, $C = 11.2\%$, $D = 41.1\%$ (expected: 25% each, χ^2=5703, $p \ll 0.001$). The recency bias (D) is the dominant effect, strongest on Flan-T5-XL ($D = 53.3\%$) and Qwen3.5-4B ($D = 54.7\%$).

Worst-selection bias: For f_{worst} (BottomUp), position D is overwhelmingly selected as "worst": D-frequencies range from 36.5% (Qwen3-4B) to 63.1% (Flan-T5-XL), far exceeding the 25% baseline. Positions B and C are severely underselected (<17% each). The pattern is consistent across all 9 models, with χ^2 values exceeding 5000 in every case. This extreme recency bias for worst-selection is the most parsimonious explanation for BottomUp's poor effectiveness: the model defaults to labeling the last passage as "worst" regardless of content.

Dual-end bias: The dual prompt produces a **novel asymmetric pattern**. For `dual_best`, the bias is substantially *reduced* compared to standalone best-selection—Qwen3-14B shows $A = 22.2\%$, $B = 26.8\%$, $C = 18.0\%$, $D = 33.1\%$, much more uniform (χ^2=1683 vs. 5703 for standalone best). For `dual_worst`, the pattern *reverses*: position A (first) becomes the most frequently selected as worst (41.3%), while D (last) is the *least* selected (12.4%). This reversal is consistent across all 9 models on both DL19 and DL20.

Table 8: Position D (last passage) selection frequency by comparison type, averaged over 9 models on DL19. The middle column shows that BottomUp (`worst`) collapses heavily onto position D, while DualEnd (`dual_worst`) does the opposite.

Type	A	B	C	D
`best`	.33	.13	.13	.41
`worst`	.24	.10	.13	.50
`dual_best`	.27	.24	.16	.33
`dual_worst`	.35	.22	.28	.13

Interpretation: The reversed `dual_worst` pattern suggests that when asked for both best *and* worst simultaneously, the model behaves differently than when asked for either signal alone. One natural reading is two-stage: the model first identifies the best (still benefiting from a recency tendency toward D), then assigns "worst" to the most salient remaining candidate—which is now typically the first passage (primacy effect). Either way, the empirical finding is that joint elicitation is not equivalent to running standalone best and standalone worst as parallel queries; the prompt structure changes the answer distribution.

Implications: The dual prompt acts as an implicit debiasing mechanism. Unlike permutation voting [17], which requires multiple passes to reduce bias, dual-end achieves bias reduction *within a single prompt*. We see this as the strongest mechanistic clue in the paper, and it directly motivates the order-robust "bias-aware DualEnd" refinement variant (§4.3), which exploits the joint-prompt asymmetry by running a small number of controlled orderings only on hard windows.

7.2 Query Difficulty Stratification

We stratify queries by difficulty (measured by BM25 NDCG@10) into Easy / Medium / Hard terciles and report ΔNDCG@10 of BottomUp-Heap and DualEnd-Cocktail against TopDown-Heap. Table 9 shows the pattern for Qwen3-14B on DL 2019, which is representative of the larger Qwen models.

Table 9: ΔNDCG@10 vs. TopDown-Heap, stratified by BM25 difficulty terciles (Qwen3-14B, DL 2019, 43 queries). DualEnd-Cocktail helps most on Medium and Hard queries; BottomUp-Heap hurts most on Hard queries.

Method	Easy	Medium	Hard	Overall
BottomUp-Heap	−.030	−.014	−.103	−.048
DualEnd-Cocktail	−.006	+.019	+.009	+.007

For Qwen3-14B, DualEnd-Cocktail gains the most on medium queries (+.019) and is roughly neutral on easy and hard queries. BottomUp-Heap is consistently negative, with the largest hurt on hard queries (−.103). For Flan-T5-XL on DL 2019, the pattern is somewhat reversed at the extremes: DualEnd-Cocktail helps on easy queries (+.014) and is mildly negative on medium and hard (−.010 / −.010), so the difficulty story should be presented as a trend conditional on model rather than as a universal rule. Across all 18 model–dataset configurations, the average DualEnd help is largest on the Easy and Medium terciles and shrinks on Hard, while BottomUp hurts roughly uniformly across terciles. Together these patterns motivate routing rather than universal use—if the gain is concentrated in particular query slices, the right place to spend DualEnd's extra calls is on those slices.

7.3 Ranking Agreement

To check whether DualEnd is essentially a perturbation of TopDown or a genuinely different ranking, we compute pairwise ranking agreement (top-10 overlap and Kendall's τ) between method pairs. Averaging over all 18 model–dataset configurations:

- TopDown vs. DualEnd: overlap@10 $\approx$ 7.0, $\tau \approx 0.93$ (high agreement).
- TopDown vs. BottomUp: overlap@10 $\approx$ 5.0, $\tau \approx 0.86$ (lower agreement).
- BottomUp vs. DualEnd: overlap@10 $\approx$ 5.2, $\tau \approx 0.88$.

Two facts follow. First, DualEnd stays mostly aligned with TopDown: it preserves the useful TopDown signal and adds refinement from the worst-selection side, rather than producing a structurally different ranking. Second, BottomUp is not random—its agreement with TopDown is still "high" on the standard tau scale—but it is consistently noisier and more divergent than DualEnd. This is exactly the regime in which fusion is unhelpful: the diverging signal does not encode complementary truth, so adding it to TopDown only adds variance, which is what we see in §6.4.

7.4 When Does DualEnd Help?

To make the "modest mean, model-dependent variance" picture concrete, we ran a per-query help/hurt summary on three representative DL19 model settings using a small qualitative analysis script that operates on the saved run files. Table 10 reports the mean delta and the help/hurt/tie counts.

Table 10: Per-query help/hurt summary for DualEnd-Cocktail vs. TopDown-Heap on three representative DL19 settings (43 queries).

Model	Mean Δ	Help	Hurt	Tie
flan-t5-xl	−0.0017	20	16	7
qwen3-8b	+0.0336	26	12	5
qwen3-14b	+0.0072	19	15	9

Three observations follow. First, DualEnd is not universally better than TopDown: on flan-t5-xl the mean delta is mildly negative even though DualEnd "wins" more queries than TopDown (20 vs. 16) than "ties" or "loses." Second, the strongest positive regime in our setting is Qwen-style causal models, especially the smaller ones. Third, on the strongest setting (qwen3-8b), most of the positive cases correspond to genuine relevant-document additions to the top-k, while on flan-t5-xl the "hurt" cases lean more toward relevant-document drops or noisy mixed changes. This directly motivates routing the joint signal where it is likely to matter rather than spending it everywhere.

7.5 Quality–Cost Pareto Frontier

We compute a Pareto analysis over the existing TREC DL run folders so the paper can talk about budgeted quality rather than isolated averages. For each method we average NDCG@10 against three cost axes (comparisons, total tokens, time per query) across all 9 models on DL19.

Table 11: Global mean Pareto frontier members on TREC DL 2019 (NDCG@10 vs. each cost axis, averaged across all 9 models). The comparison- and token-frontier contains only three points: TopDown-Heap, TopDown-Bubble, and DualEnd-Cocktail.

Frontier	Members
Comparisons	TD-Heap, TD-Bubble, DE-Cocktail
Total tokens	TD-Heap, TD-Bubble, DE-Cocktail
Time	PermVote(p=2), TD-Heap, TD-Bubble, DE-Cocktail

Frontier members and means (mean NDCG@10 / mean comparisons / mean total tokens / mean time):

- TopDown-Heap: .6851 / 76.5 / 32464 / 28.41 s
- TopDown-Bubble: .6897 / 300.0 / 126456 / 110.93 s
- DualEnd-Cocktail: .6962 / 546.0 / 233192 / 212.62 s

Interpretation. TopDown-Heap remains the cheap anchor; TopDown-Bubble is the strongest middle-cost anchor; DualEnd-Cocktail is the quality-first anchor. Crucially, the global frontier contains *no* BottomUp or BiDir variant on either the comparisons or the tokens axis. BottomUp is dominated because it spends many more comparisons than TopDown-Heap at strictly worse mean quality; BiDir is dominated for the same reason—it inherits BottomUp's noise and cost. The time-axis frontier additionally admits permutation voting (p=2) as a fast-but-weaker point; the quality-axis story does not change. This identifies the paper's refinement target precisely: the empty region between TopDown-Bubble and DualEnd-Cocktail. The Selective DualEnd, bias-aware DualEnd, and same-call worst-signal regularization variants in §4.3 are designed to land in that region rather than to push the upper-right corner further.

7.6 Error Analysis

We close with a brief qualitative error analysis to characterize what each method gets wrong.

Top-down errors: TopDown tends to make errors when multiple documents are similarly relevant, and position bias causes the LLM to consistently favor certain positions. This leads to "promoted impostors"—documents that are ranked higher than they deserve because they were systematically selected as best when placed in favorable positions.

Bottom-up errors: BottomUp tends to make errors when documents are similarly *ir*relevant, and the LLM struggles to identify the "most irrelevant" among a set of non-relevant documents. Combined with the strong recency bias for the standalone worst-selection prompt, this leads to "survivor bias"—non-relevant documents that survive elimination and contaminate the top-k.

Dual-end errors: DualEnd introduces a new error type: *interference errors*, where the simultaneous best-and-worst assessment causes the LLM to produce an inconsistent pair (e.g., selecting the same document as both best and worst). Our "same best&worst" warning count shows this occurs in fewer than 0.1% of dual comparisons (1–2 instances per run), indicating interference is rare. More common are partial parse failures where only one label is extracted (<1% of comparisons), handled by heuristic defaults.

Contrary to our original hypothesis, these complementary error patterns do *not* make BiDir fusion effective. The BottomUp errors are too frequent and systematic (driven by position bias, not by random noise) to cancel out TopDown errors. DualEnd, in contrast, naturally combines both selection directions in a single call, achieving the error-cancellation benefit without the fusion overhead.

7.7 Relationship to Tournament Theory

Our dual-end selection sort can be viewed through the lens of tournament theory. In a standard single-elimination tournament, each match produces one winner and one loser. The winner advances; the loser is discarded. This is analogous to standard setwise ranking. A *double-elimination tournament* tracks both the winners' bracket and the losers' bracket, giving each competitor two chances before elimination. Our DualEnd-Selection is similar: each comparison round identifies both bracket winners and bracket losers, and both results are used for ranking. Recent work on tournament graphs for LLM ranking [1] also seeks to extract richer information from

each comparison, but through graph-theoretic analysis rather than prompt-level modifications.

The information-theoretic advantage of dual-end (∼74–90% more bits per call depending on window size) provides a theoretical upper bound on the efficiency improvement. In practice, the realized improvement depends on: (1) the accuracy of worst-selection relative to best-selection, (2) the overhead of parsing dual outputs, and (3) whether the sorting algorithm can fully exploit both outputs. Our empirical results in §6.3 confirm the practical gain is real but modest—enough to make DualEnd the strongest family overall, not enough to claim Bonferroni-significant wins on most TREC DL configurations.

8 Limitations

Our approach has several limitations that should be acknowledged.

Statistical fragility on small TREC DL test sets: With 43 (DL19) and 54 (DL20) queries, only one DualEnd configuration (Qwen3-4B DL19) survives Bonferroni correction with $p_{\text{Bonf}} = 0.010$. The remaining 13 positive DualEnd configurations are directionally consistent but cannot be claimed as significant on these query budgets. By contrast, the negative results—6 Bonferroni-significant BottomUp losses and 3 BiDir losses—are statistically more robust. We have therefore framed the headline contribution as "directional asymmetry" rather than "DualEnd is universally significantly better."

Efficiency of bottom-up for top-k: BottomUp requires $\sim n-1$ extractions for top-k ranking, making it ∼3.5× more expensive than TopDown-Heap when $k = 10$, $n = 100$ (and BottomUp-Bubble is more than 20× more expensive). This limits its use as a standalone method; it is primarily valuable as a diagnostic tool for the directional asymmetry argument.

DualEnd cost: DualEnd-Cocktail uses about 7× more comparisons than TopDown-Heap and about 1.7× more than TopDown-Bubble. The token-level overhead is similar. We do not claim end-to-end efficiency gains; we claim more useful bits per call. The Pareto analysis in §7.5 shows that the global mean frontier contains only TopDown-Heap, TopDown-Bubble, and DualEnd-Cocktail, and this is the right way to read the cost story.

Likelihood for worst-related signals remains heuristic: For Flan-T5 we use direct logits over single-character labels. For Qwen3/Qwen3.5 we use a teacher-forced short-answer scheme (`Passage A`, etc.) that is robust to causal tokenization. For *dual-end* likelihood we reuse the best-only distribution and take its maximum and minimum as the best/worst outputs. This is useful as an efficiency probe and matches the heuristic Flan-T5 path, but it is not an exact likelihood model of the full "Best: X, Worst: Y" string. The same caveat applies to the routed joint-signal refinements in §4.3 whenever they enter their joint-signal path under `-scoring likelihood`.

Parsing fragility: Dual-end generation requires the LLM to produce structured output with two labels. While our cascading parser handles most formats, the success rate varies across models. Models that tend to produce verbose explanations rather than concise labels may have lower parsing success rates. The worst case in our experiments is Qwen3-14B on DL 2020 with a 0.9% partial parse rate (handled by heuristic defaults).

Model dependence: Our experiments cover three architecture families (encoder-decoder, decoder-only, and hybrid causal) and nine models ranging from 780M to 27B parameters. Results may differ for larger models (e.g., 70B+) or API-based models (GPT-4, Claude), which may have stronger format-following abilities and different position bias patterns.

Dataset scope: Our headline results come from TREC DL 2019/2020. We are also running a representative BEIR subset on Flan-T5-XL, Qwen3-8B, and Qwen3.5-9B; those runs are still completing and will be folded into the camera-ready version. All current benchmarks use relatively short passages, and performance on longer documents or domain-specific corpora with very long documents remains to be validated.

Refinement variants are implemented but not yet fully evaluated: The Selective DualEnd, bias-aware DualEnd, and same-call worst-signal regularization variants in §4.3 are runnable code paths in our `llm-rankers` fork, with their own logged counters. We use them in this paper as a motivated next-step package rather than as a primary empirical contribution; their full evaluation is reserved for follow-up work.

9 Conclusion

We introduced three bidirectional strategies for LLM-based setwise document ranking that challenge the convention of extracting only "best" information from each LLM comparison. Our experiments with nine LLMs across three architecture families yield three findings.

First, **the DualEnd family is the strongest overall**. DualEnd-Cocktail achieves the highest average NDCG@10 across all 9 models on both DL 2019 (.7134) and DL 2020 (.6791), and DualEnd methods (Cocktail or Selection) win in 14 of 18 model–dataset configurations. The per-query significance picture is more conservative: only one of those gains remains Bonferroni-significant. By asking the LLM for both the best and worst document per call, dual-end methods narrow the ranking from both ends, and an unexpected benefit is *implicit debiasing*: dual-end prompting produces more uniform position selection patterns than single-direction methods, partially mitigating the strong recency bias observed in standalone best- and worst-selection.

Second, **bottom-up ranking is substantially weaker than top-down**, refuting the hypothesis that LLMs are better at identifying irrelevant documents. The gap is particularly severe for smaller models (Flan-T5-Large: .289 vs. .654 NDCG@10) and narrows but does not vanish at scale. Bonferroni-significant losses appear in 6 of 18 configurations. Position bias analysis reveals a likely mechanism: models exhibit extreme recency bias when selecting "worst" (position D selected 40–63% of the time), effectively defaulting to the last passage.

Third, **bidirectional ensemble does not outperform the top-down baseline**, indicating that simply fusing top-down and bottom-up rankings cannot compensate for the weakness of the bottom-up component. Even at $\alpha = 0.9$ (90% TopDown weight), fusion underperforms pure TopDown, and BiDir produces 3 Bonferroni-significant losses. The intuitive assumption that directional fusion would parallel the success of permutation voting [17] does not hold because the directions are not equally reliable.

The strongest scientific contribution is therefore one of *directional asymmetry*: worst-selection alone is not the dual of best-selection, and the gap between standalone worst-selection and joint best-worst elicitation is mechanistically meaningful. The strongest practical contribution is the empirical map that says which sorting algorithm and which selection direction works for which model family, and the quality–cost Pareto frontier that pinpoints the missing intermediate point between TopDown-Bubble and DualEnd-Cocktail.

Our findings open several avenues for future work: (1) full empirical evaluation of the Selective, bias-aware, and same-call regularized DualEnd refinement variants, all of which are already implemented and instrumented in our fork; (2) extending dual-end to extract a *full ordering* of the candidate set, effectively bridging setwise and listwise paradigms; (3) investigating whether reasoning-enhanced rerankers [22] benefit from dual-end prompting; (4) benchmarking the new Qwen/Qwen3.5 likelihood follow-up against the generation results to quantify the cost saved and the quality lost; and (5) understanding why worst-selection is harder for LLMs and whether this can be mitigated through chain-of-thought reasoning.

References

[1] Aman Agrawal et al. 2026. BlitzRank: Principled Zero-shot Ranking Agents with Tournament Graphs. *arXiv preprint arXiv:2602.05448* (2026).

[2] Yiqun Chen, Qi Liu, Yi Zhang, Weiwei Sun, Xinyu Ma, Wei Yang, Daiting Shi, Jiaxin Mao, and Dawei Yin. 2025. TourRank: Utilizing Large Language Models for Documents Ranking with a Tournament-Inspired Strategy. In *Proceedings of the ACM Web Conference 2025*.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 758–759.

[4] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and Daniel Campos. 2021. Overview of the TREC 2020 Deep Learning Track. In *Proceedings of TREC 2020*.

[5] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. 2020. Overview of the TREC 2019 Deep Learning Track. In *Proceedings of TREC 2019*.

[6] Donald E. Knuth. 1997. *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.

[7] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. 2021. Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations. *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2021), 2356–2362.

[8] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

[9] Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. 2023. Zero-Shot Listwise Document Reranking with a Large Language Model. *arXiv preprint arXiv:2305.02156* (2023).

[10] Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. 2020. Document Ranking with a Pretrained Sequence-to-Sequence Model. *Findings of the Association for Computational Linguistics: EMNLP 2020* (2020), 708–718.

[11] Jakub Podolak, Leon Peric, Mina Janicijevic, and Roxana Petcu. 2025. Beyond Reproducibility: Advancing Zero-shot LLM Reranking Efficiency with Setwise Insertion. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

[12] Ronak Pradeep, Rodrigo Nogueira, and Jimmy Lin. 2021. The Expando-Mono-Duo Design Pattern for Text Ranking with Pretrained Sequence-to-Sequence Models. *arXiv preprint arXiv:2101.05667* (2021).

[13] Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2024. Large Language Models are Effective Text Rankers with Pairwise Ranking Prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 1504–1518.

[14] Devendra Singh Sachan, Mike Lewis, Mandar Joshi, Armen Aghajanyan, Wen-tau Yih, Joelle Pineau, and Luke Zettlemoyer. 2022. Improving Passage Retrieval with Zero-Shot Question Generation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

[15] Ryoma Sato. 2026. A Survey on Sorting with Large Language Models. *DPC Technical Report DPC-TR-2026-001* (2026).

[16] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14009–14026.

[17] Raphael Tang, Xinyu Zhang, Xueguang Ma, Jimmy Lin, and Ferhan Ture. 2024. Found in the Middle: Permutation Self-Consistency Improves Listwise Ranking in Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 511–524.

[18] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of the 35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[19] Oren Wisznia et al. 2025. Are Optimal Algorithms Still Optimal? Rethinking Sorting for LLM-based Pairwise Ranking. *arXiv preprint arXiv:2505.24643* (2025).

[20] Yifan Zeng, Ojas Tendolkar, Raymond Baartmans, Qiufeng Wu, Liyu Chen, and Huazheng Wang. 2024. LLM-RankFusion: Mitigating Intrinsic Inconsistency in LLM-based Ranking. *arXiv preprint arXiv:2406.00231* (2024).

[21] Xinyu Zhang, Sebastian Hofstatter, Patrick Lewis, Raphael Tang, and Jimmy Lin. 2025. Rank-without-GPT: Building GPT-Independent Listwise Rerankers on Open-Source Large Language Models. *Proceedings of the 47th European Conference on Information Retrieval* (2025).

[22] Shengyao Zhuang, Xueguang Ma, Bevan Koopman, Jimmy Lin, and Guido Zuccon. 2025. Rank-R1: Enhancing Reasoning in LLM-based Document Rerankers via Reinforcement Learning. *arXiv preprint arXiv:2503.06034* (2025).

[23] Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. 2024. A Setwise Approach for Effective and Highly Efficient Zero-shot Ranking with Large Language Models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 38–47.